

智能移动开发7

前提条件

- 熟悉 Kotlin 语言，包括语法、函数和变量。
- 能够在 Compose 中构建布局，包括带内边距的行和列。
- 能够在 Compose 中创建简单列表。

学习内容

- 如何将 Material 主题设置应用于 Compose 应用。
- 如何为您的应用添加自定义调色板。
- 如何为您的应用添加自定义字体。
- 如何向应用中的元素添加自定义形状。
- 如何向应用中添加顶部应用栏。

增加的导入包

```
import androidx.compose.ui.layout.ContentScale
```

```
import androidx.compose.material3.Scaffold
```

学习截图

1 准备工作

2 应用概览

3 获取起始代码

4 添加颜色

5 添加形状

6 添加排版

7 添加顶部栏

8 获取解决方案代码

9 总结

报告错误

4. 依次打开 `res > values > strings.xml`。此文件包含您在此应用中使用的字符串，包括应用名称、狗狗名字、说明等。

5. 打开 `MainActivity.kt`。此文件包含创建简单列表的代码，用于显示狗狗的照片、名字及年龄。

6. `WoofApp()` 包含用于显示 `DogItem` 的 `LazyColumn`。

7. `DogItem()` 包含用于显示狗狗照片及相关信息的 `Row`。

8. `DogIcon()` 显示小狗的照片。

9. `DogInformation()` 显示狗狗的名字和年龄。

10. `WoofPreview()` 可让您在 `Design` 窗格中预览应用。

注意：您可能会注意到，每个 `@Composable` 方法都有一个修饰符作为参数。在 Compose 中，最佳实践是将修饰符参数传递给可组合函数。这是因为它允许父级可组合项将上下文信息传递给子级可组合项。

例如，假设有一个按钮，一个父按钮可能想让其子按钮使用完整尺寸，而另一个父按钮可能想让孩子按钮封装内容。这样会提高代码的可重用性。此参数通常名为“修饰符”，应作为函数的参数列表的第一个可选参数显示。此修饰符将应用于方法的第一个子级。如需了解详情，请参阅 [Jetpack Compose 的 API 准则](#)。

确保您的模拟器/设备采用浅色主题

在 [返回](#) 中，您将使用浅色主题和深色主题，不过，此 Codelab 中的大部分主题都采用浅色主题。之前，请确保您的设备/模拟器使用的是浅色主题。

下一步

在 Android 系统中，颜色用十六进制颜色值表示。十六进制颜色代码以井号 (#) 字符开头，后跟六个字母和/或数字（代表该颜色的红色、绿色和蓝色 [RGB] 分量）。前两个字母/数字表示红色，后面的两个表示绿色，最后两个表示蓝色。

颜色还可以包含 Alpha 值（字母和/或数字），用于表示颜色的透明度（#00 表示不透明度为 0% [完全透明]，#FF 表示不透明度为 100% [完全不透明]）。若添加 alpha 值，则该值为井号 (#) 字符后的十六进制颜色代码的前两个字符。如果未添加 alpha 值，系统会假定它是 #FF，即 100% 不透明（完全不透明）。

Material Theme Builder

Core colors

Override or set key colors that will be used to generate tonal palettes and colors.

Color match

Stay true to my color inputs.

Primary

Acts as custom source color

Secondary

Tertiary

Error

Neutral

1 of 2

Add Theme Colors

HCT Color Picker

Hex Color

#006C4C

Hue

166.70248

Chroma

40.591163

Tone

39.993284

Cancel

Apply

Filter

Filter

Textfield

Button

+ Button

Button

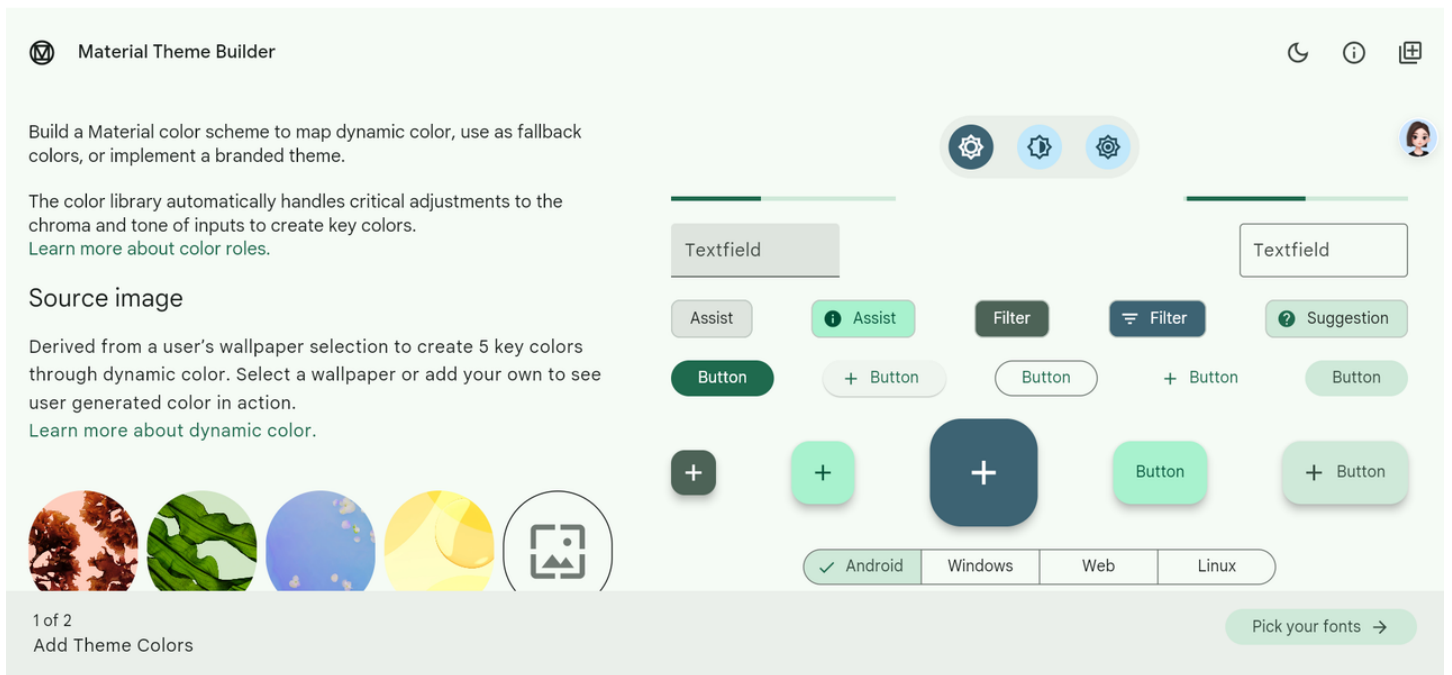
+ Button

Windows

Web

Linux

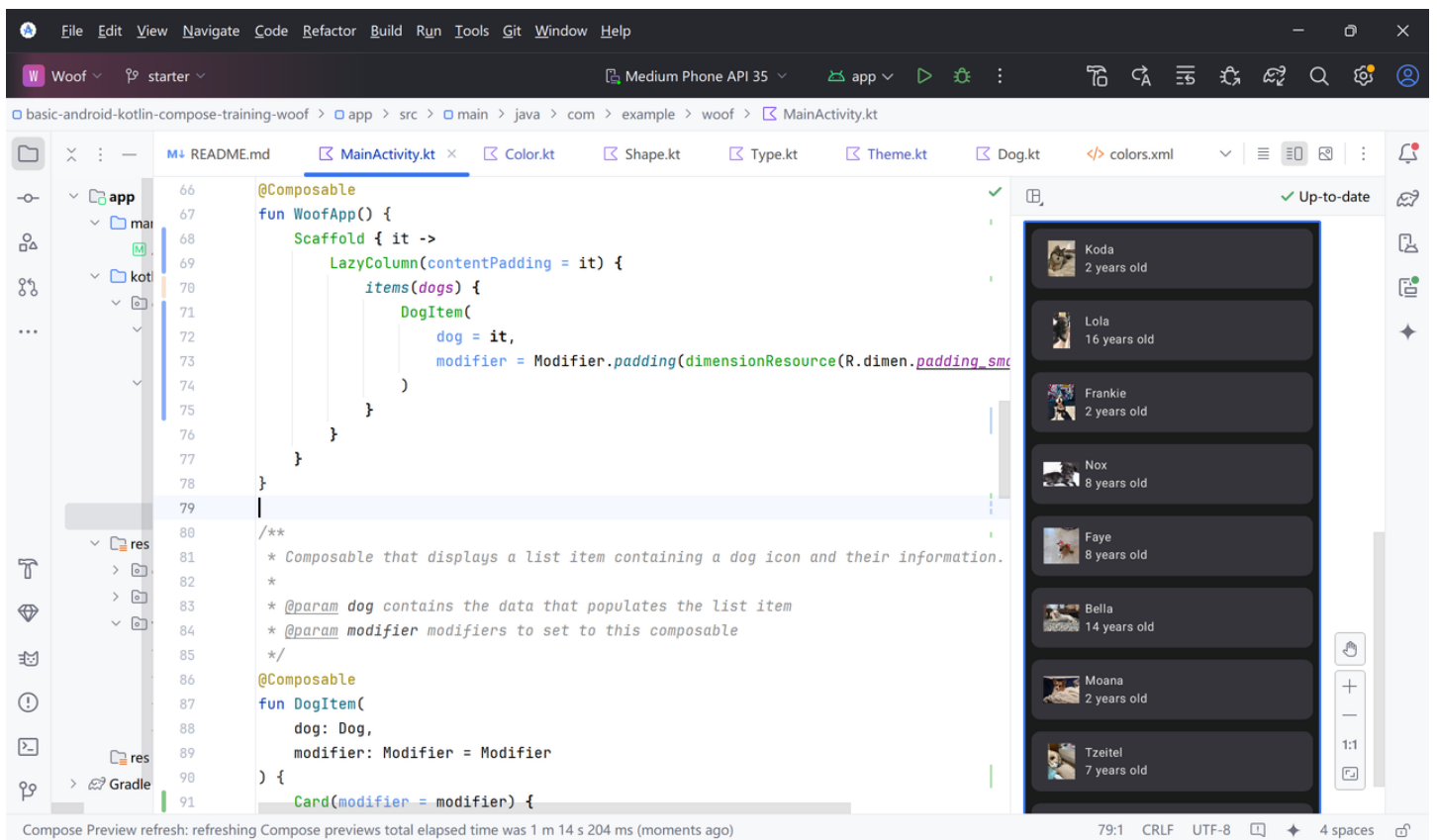
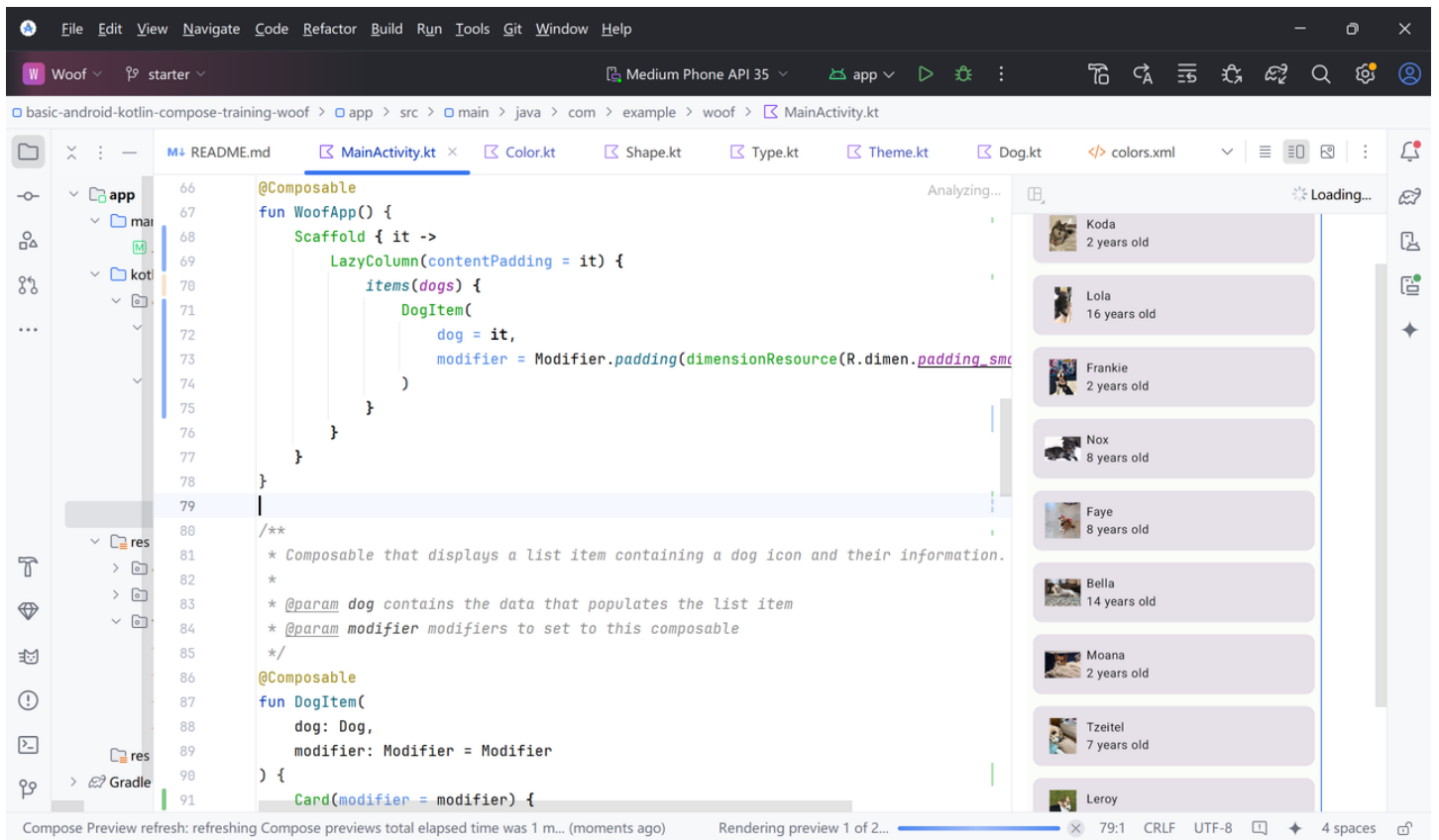
Pick your fonts →



您可能想了解所有这些角色的含义及其使用方法，以下是其中几个主要角色：

- primary（主色）用于整个界面的关键组件。
- secondary（副色）用于界面中不太显眼的组件。
- tertiary（第三色）用于对比强调，可以平衡主色和副色，或者引起用户对某个元素（例如输入字段）的高度关注。
- on 颜色元素显示在调色板中其他颜色的上层，主要应用于文本、图标和描边。在调色板中，我们有一个 onSurface 颜色（显示在 surface 颜色的上层）和一个 onPrimary 颜色（显示在主要颜色的上层）。

有了这些槽可以实现统一的设计体系，相关组件的颜色也相似。



```

@Composable
fun WoofTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
    }
}

```

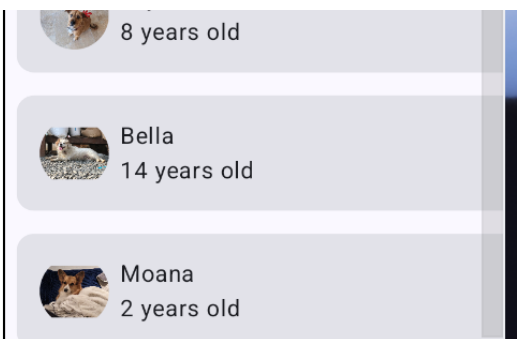
5. Add shape

the dog icons are circular

```

(
    modifier = modifier
        .size(dimensionResource(R.dimen.image_size))
        .padding(dimensionResource(R.dimen.padding_small))
        .clip(MaterialTheme.shapes.small),
    painter = painterResource(dogIcon),
)

```



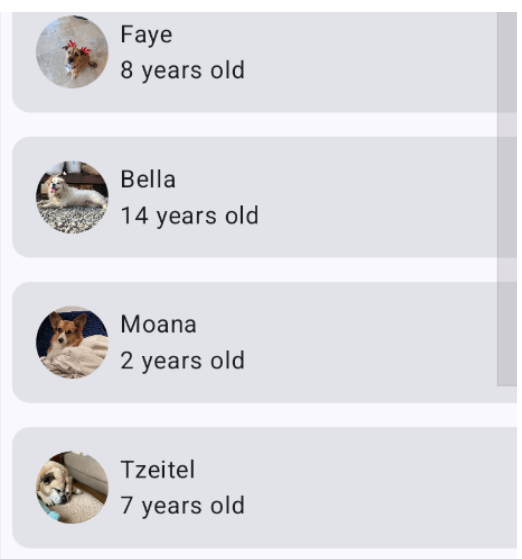
make all the photos circular

```

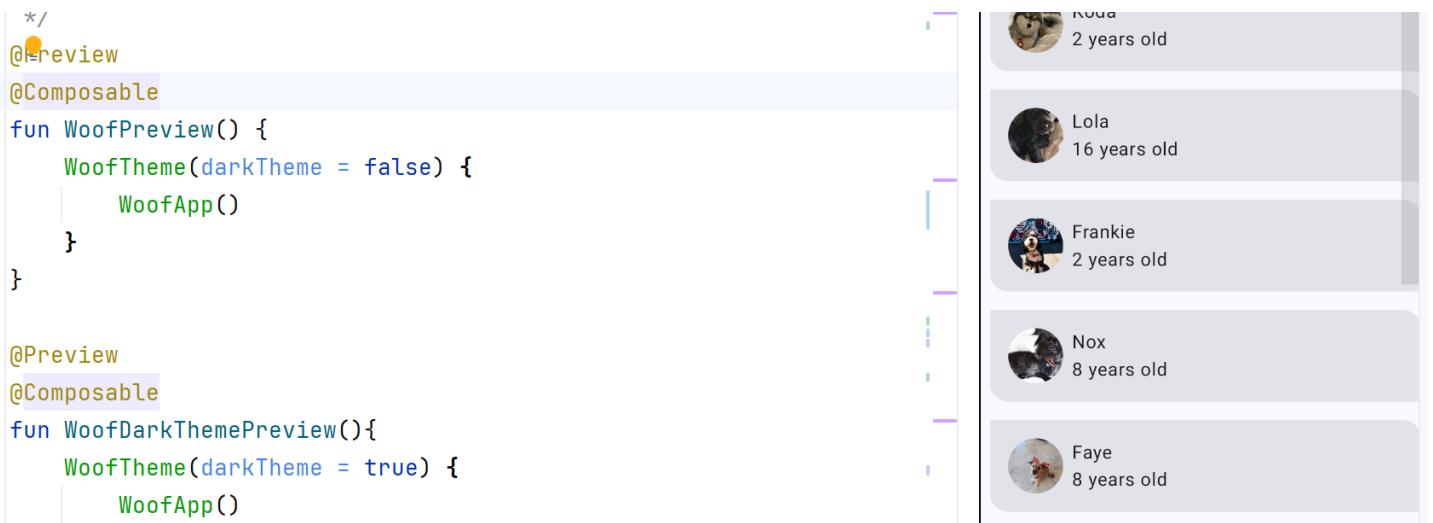
) {
    Image(
        modifier = modifier
            .size(dimensionResource(R.dimen.image_size))
            .padding(dimensionResource(R.dimen.padding_small))
            .clip(MaterialTheme.shapes.small),
        contentScale = ContentScale.Crop,
        painter = painterResource(dogIcon),

        // Content Description is not needed here
        // description allows accessibility service
    )
}

```



Add a shape to the list item



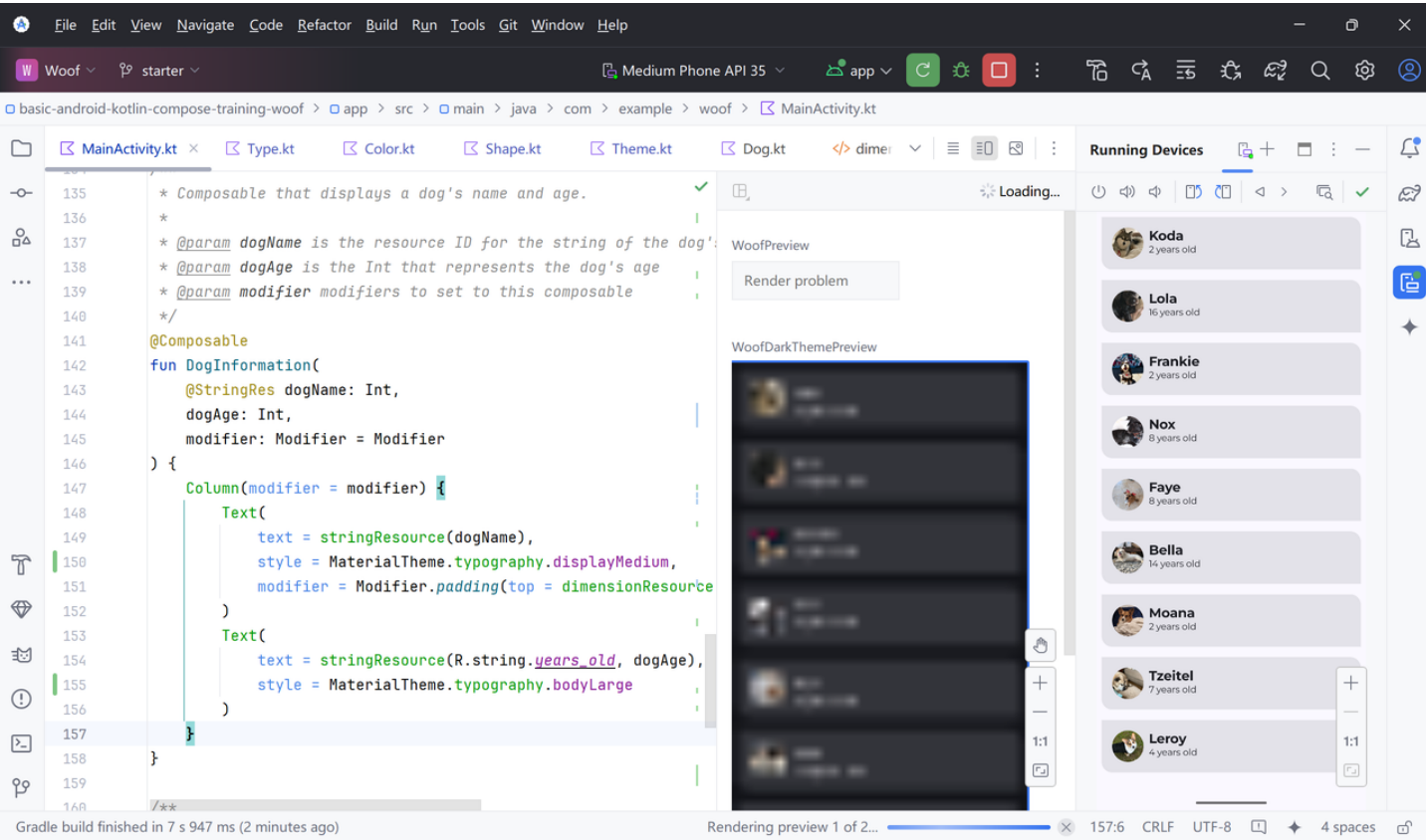
6. Add typography

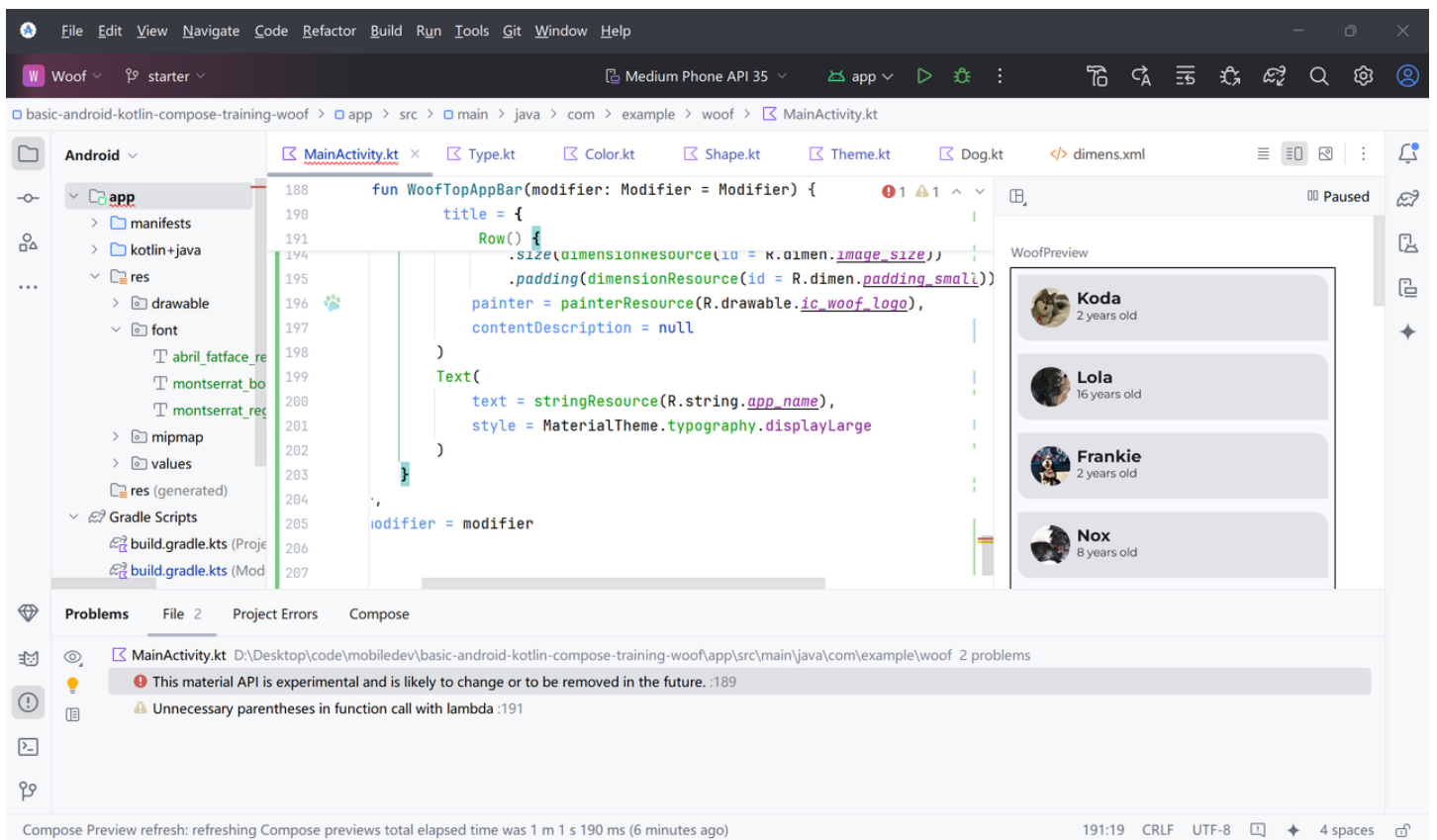
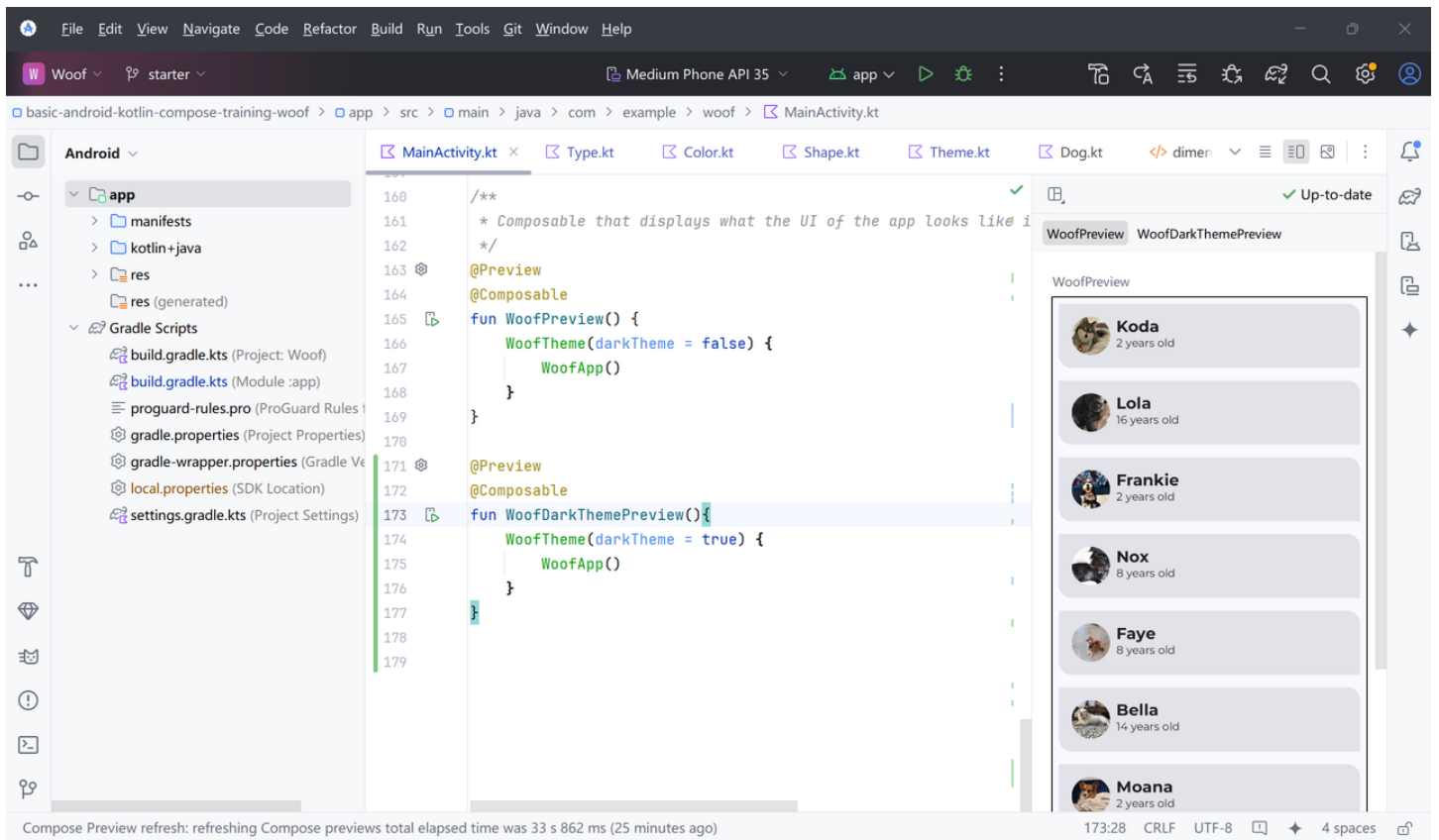
The type scale contains reusable categories of text, each with an intended application and meaning.

rename Montserrat-Bold.ttf to montserrat_bold.ttf and rename Montserrat-Regular.ttf to montserrat_regular.ttf

rename Abril_Fatface-Regular.ttf to abril_fatface_regular.ttf.

Add typography to app text





问题

opAppBar显示不出来，不知道哪里出错了，MainActivity文件和解决方案里一样但显示不出来，更改gradle也尝试了，也没成功